

Sensors & Systems — Lecture 7 Notes

Ranging-based Sensors: LiDAR, Radar, Sonar

Jan Bendtsen, Torben Knudsen | Electronic Systems, 8th Semester

Sensor Overview — Where to Use What

Sensor	Wave	Range	Resolution	Best use / weakness
Sonar	Sound	Medium (water)	Moderate	Underwater only; radio/light absorbed in water
Radar	Radio (mm–cm)	Long (km)	Low–Med	All weather, measures radial velocity via Doppler; poor spatial resolution
LiDAR	Laser (near-IR)	Short–Med	Very high	Sub-cm point clouds; fails in fog/rain; no colour

Sensor fusion is used when no single sensor covers all needs. Autonomous vehicles combine all three: Radar handles adverse weather and measures velocity; LiDAR maps 3D geometry precisely; Camera reads colour, text, and signs. Each fills the blind spots of the others. Where only one medium is viable (e.g. underwater), a single sensor suffices — active sonar is the only practical ranging method in water.

Pulsed vs. Phased LiDAR

Both methods exploit the finite speed of light $c \approx 3 \times 10^8 \text{ m/s}$ but extract distance differently.

Pulsed LiDAR — time of flight

Fire a short pulse ($< 50 \text{ ns}$), start a timer, stop when echo returns. Round-trip covers $2R$, so:

$$R = \frac{1}{2} c t$$

Distance resolution is set by the clock interval Δt :

$$\Delta R = \frac{1}{2} c \Delta t \quad \text{e.g. } \Delta t = 1 \text{ ns} \Rightarrow \Delta R = 15 \text{ cm}$$

Phased (continuous-wave) LiDAR — phase comparison

Emit a sinusoidally modulated beam at frequency f . The round-trip delay appears as a phase shift φ :

$$\frac{t}{T} = \frac{\varphi}{2\pi} \implies R = \frac{1}{2}c \frac{T\varphi}{2\pi} = \frac{c}{4\pi f} \varphi = \frac{\lambda}{4\pi} \varphi$$

where $\lambda = c/f$ is the *modulation* wavelength (not the laser's optical wavelength). Maximum unambiguous range is $R_{\max} = \lambda/2$ — beyond this the phase wraps and distances become indistinguishable.

	Pulsed	Phased
Range from	Time of flight t	Phase shift φ
Formula	$R = \frac{1}{2}ct$	$R = \frac{\lambda}{4\pi}\varphi$
Range limit	Laser power / atmosphere	Modulation wavelength ($\lambda/2$)
Typical range	Long (> 100 m)	Shorter (~ 1 – 100 m)
Precision	Moderate ($\sim$ cm)	Higher (sub-mm possible)

Why phased LiDAR is more precise. Pulsed systems measure nanosecond time intervals — 1 ns clock gives only $\Delta R = 15$ cm. Phased systems measure a phase angle, which electronics can resolve to fractions of a degree, giving sub-mm precision at short range. The cost is the limited unambiguous range $\lambda/2$.

Design rule for phased LiDAR: choose modulation frequency f so that $\lambda/2$ exceeds the furthest expected target:

$$f < \frac{c}{2 R_{\max, \text{expected}}}$$

Lower $f \rightarrow$ longer $\lambda \rightarrow$ larger range but coarser phase ruler $\rightarrow$ worse precision. This is the core trade-off.

Ranging Precision

Precision is the standard deviation σ of the distance estimate. It scales with the signal-to-noise ratio (SNR):

$$\sigma_{\text{pulse}} \propto \frac{c}{2} t_{\text{rise}} \sqrt{\frac{B_{\text{pulse}}}{P_{\text{peak}}}}, \quad \sigma_{\text{phase}} \propto \frac{\lambda}{4\pi} \sqrt{\frac{B_{\text{cw}}}{P_{\text{av}}}}$$

Symbol	Name	Effect on precision
t_{rise}	Rise time of pulse	Shorter edge $\rightarrow$ sharper timing $\rightarrow$ better
λ	Modulation wavelength	Shorter $\rightarrow$ finer phase ruler $\rightarrow$ better
B	Noise bandwidth	Wider $\rightarrow$ more noise $\rightarrow$ worse
P	Received power	More power $\rightarrow$ better SNR $\rightarrow$ better

Key insight: $\sigma \propto \sqrt{B/P}$ — to halve σ you need to quadruple received power or quarter the noise bandwidth.

LiDAR Scattering Model — Received Power

The received power follows a four-step chain from laser to receiver:

$$P_R = P_t \cdot \underbrace{\frac{A_S}{A_{\text{laser}}}}_{\text{fraction hitting target}} \cdot \underbrace{\rho}_{\text{reflectance}} \cdot \underbrace{\frac{1}{\Omega R^2} \cdot \frac{\pi D_r^2}{4}}_{\text{fraction captured by receiver}} \cdot \underbrace{\eta_{\text{atm}} \eta_{\text{sys}}}_{\text{efficiency losses}}$$

Substituting the footprint area $A_{\text{laser}} = \pi R^2 \beta_t^2 / 4$ (which grows as R^2) gives the full range equation:

$$P_R = \frac{4P_t}{\pi R^2 \beta_t^2} \cdot \rho A_S \cdot \frac{1}{\Omega R^2} \cdot \frac{\pi D_r^2}{4} \cdot \eta_{\text{atm}} \eta_{\text{sys}}$$

Symbol	Name	Meaning
P_t	Transmitted power	Total laser pulse power
R	Range	Distance to target
β_t	Beam divergence	Half-angle of emitted cone; wider = larger footprint
A_S	Scatterer area	Effective reflecting area of target
ρ	Reflectance	Fraction of incident power reflected (0–1)
Ω	Solid angle	Cone into which target disperses power
D_r	Receiver diameter	Aperture of sensor lens
η_{atm}	Atmospheric efficiency	Loss from absorption/scattering in air
η_{sys}	System efficiency	Loss in sensor optics and electronics

The R^4 penalty. $P_R \propto 1/R^4$ — one $1/R^2$ from the footprint growing (less power hits the target) and another $1/R^2$ from the scattered power spreading before the receiver catches its slice. Doubling range reduces returned power by **16×**. This is why long-range LiDAR requires powerful lasers and sensitive detectors.

Noise Model for LiDAR Measurements

Each point cloud measurement $(d_{i,k}, \alpha_{i,k})$ is affected by independent zero-mean Gaussian noise:

$$\tilde{S}_k = (\bar{d}_{i,k} + \nu_{d,k}, \bar{\alpha}_{i,k} + \nu_{\alpha,k}), \quad \nu_{d,k} \sim \mathcal{N}(0, \sigma_{\nu_d}^2), \quad \nu_{\alpha,k} \sim \mathcal{N}(0, \sigma_{\nu_\alpha}^2)$$

Noise is modelled in polar coordinates because angle noise scales with distance while range noise is constant:

$$\Delta_{\text{tangential}} = \bar{d}_i \cdot \sin(\nu_\alpha) \approx \bar{d}_i \cdot \nu_\alpha$$

Why angle noise is more dangerous at long range. A 1° angle error causes a lateral shift $\Delta = d \cdot \sin(1^\circ)$:

$$d = 1 \text{ m} : \quad \Delta \approx 1.7 \text{ cm} \quad d = 10 \text{ m} : \quad \Delta \approx 17 \text{ cm}$$

Range noise σ_{ν_d} is constant: 5 cm at 1 m is 5 cm at 10 m.

Uncertainty Ellipse: Polar to Cartesian

The LRF measures polar coordinates; downstream algorithms work in Cartesian. The conversion is nonlinear:

$$\xi_i = \begin{bmatrix} x_i \\ y_i \end{bmatrix} = \begin{bmatrix} \bar{d}_i \cos \bar{\alpha}_i \\ \bar{d}_i \sin \bar{\alpha}_i \end{bmatrix}$$

The polar noise covariance (range and angle independent):

$$\Sigma = \begin{bmatrix} \sigma_{\nu_d}^2 & 0 \\ 0 & \sigma_{\nu_\alpha}^2 \end{bmatrix}$$

To propagate Σ into Cartesian space we linearise using the Jacobian (same idea as the EKF):

$$J(\bar{d}_i, \bar{\alpha}_i) = \frac{\partial[x, y]}{\partial[d, \alpha]} = \begin{bmatrix} \cos \bar{\alpha}_i & -\bar{d}_i \sin \bar{\alpha}_i \\ \sin \bar{\alpha}_i & \bar{d}_i \cos \bar{\alpha}_i \end{bmatrix}$$

The Cartesian covariance is then:

$$\text{Cov}_{\text{Cartesian}} \approx J \Sigma J^\top$$

The confidence ellipsoid in polar coordinates (probability level $1 - p$):

$$\frac{(\bar{d}_i - d_i)^2}{\sigma_{\nu_d}^2} + \frac{(\bar{\alpha}_i - \alpha_i)^2}{\sigma_{\nu_\alpha}^2} = \chi_n^2(1 - p)$$

After the Jacobian transform the ellipse becomes tilted in Cartesian space:

- **Radial axis** (along beam): half-length $\approx \sigma_{\nu_d} \sqrt{\chi^2}$ — constant, does not grow with range.
- **Tangential axis** (perpendicular to beam): half-length $\approx \bar{d}_i \sigma_{\nu_\alpha} \sqrt{\chi^2}$ — grows linearly with range.

At short range the ellipse is compact in all directions. At long range it becomes elongated sideways: you know the distance well but the lateral position is increasingly uncertain — the tangential axis keeps growing while the radial axis stays fixed.

Circle (Disc) Fitting — Maximum Likelihood

Each robot is modelled as a disc. Given N_K noisy Cartesian points $\xi_{i,k}$ in cluster K , the best-fitting disc centre $\xi_{c,k} = (x_{c,k}, y_{c,k})$ and radius ρ minimise:

$$\min_{\xi_{c,k}, \rho} \sum_{i=0}^{N_K} \left[(\xi_{c,k} - \xi_{i,k})^\top (\xi_{c,k} - \xi_{i,k}) - \rho^2 \right]^2$$

This is the maximum likelihood estimate when the points have equal, isotropic Gaussian noise — it finds the centre and radius most consistent with all N_K noisy measurements simultaneously. Noise on different points is independent, so errors partly cancel and the centre estimate is much more reliable than any single point alone.

Reformulation as linear least squares

The nonlinear circle equation $(x_i - x_c)^2 + (y_i - y_c)^2 = \rho^2$ is expanded and rearranged. Define:

$$p_1 = 2x_{c,k}, \quad p_2 = 2y_{c,k}, \quad p_3 = \rho^2 - x_{c,k}^2 - y_{c,k}^2$$

Each point then gives a *linear* equation in (p_1, p_2, p_3) :

$$p_1 \cdot x_{i,k} + p_2 \cdot y_{i,k} + p_3 \cdot 1 = x_{i,k}^2 + y_{i,k}^2$$

Stacking all N_K equations: $X^\top p = q$, solved by the normal equation:

$$p = (XX^\top)^{-1}Xq \quad \text{where} \quad X = \begin{bmatrix} x_{1,k} & x_{2,k} & \cdots & x_{N_K,k} \\ y_{1,k} & y_{2,k} & \cdots & y_{N_K,k} \\ 1 & 1 & \cdots & 1 \end{bmatrix}, \quad q = \begin{bmatrix} x_{1,k}^2 + y_{1,k}^2 \\ x_{2,k}^2 + y_{2,k}^2 \\ \vdots \\ x_{N_K,k}^2 + y_{N_K,k}^2 \end{bmatrix}$$

Extract the result: $x_{c,k} = p_1/2$, $y_{c,k} = p_2/2$, $\rho = \sqrt{p_3 + x_{c,k}^2 + y_{c,k}^2}$.

$XX^\top$ is only 3×3 regardless of how many points there are — so the inversion is trivial. This gives the global minimum in one step with no iteration and no initial guess needed.

Compare with line fitting (Exercise 6): the same idea applies. For a line $ax + by + c = 0$ we minimise $\sum (ax_i + by_i + c)^2 / (a^2 + b^2)$ — perpendicular distances to the line. The circle fitting minimises radial distances to a circle boundary instead. Both reduce to a linear normal equation after the right substitution.

Kalman Filter for Robot Tracking

The disc centre $\xi_{c,k}$ from circle fitting is fed into a Kalman filter as the position measurement. With no knowledge of robot dynamics, a constant-velocity model is used:

$$\mathbf{x}_k = \begin{bmatrix} x_k \\ y_k \\ v_{x,k} \\ v_{y,k} \end{bmatrix}, \quad \Phi = \begin{bmatrix} 1 & 0 & T_s & 0 \\ 0 & 1 & 0 & T_s \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

H extracts only position since the LiDAR measures disc centres, not velocity. The KF infers velocity from how position changes between steps. For multiple robots, independent filter instances run in parallel — one state vector per robot.

Exercises: We do All 4, Present Only 4–6

3. Write a Python function that determines if a straight line intersects a specified circle, returning the intersection coordinates.
4. Use the function above to emulate a LiDAR shooting at a cylindrical robot as in Wasik et al. Plot the robot and the point cloud. Add noise to the measurements.
5. Place the simulated robot in front of a wall. Simulate the point cloud from both robot and wall.
6. Assign wall point cloud data to one cluster. Use the optimisation from Ex. 2 to determine the equation of the corresponding line.

Exercise 4: LiDAR point cloud on cylindrical robot

For each beam angle $\alpha_i \in [0, 2\pi]$ we check whether the ray from the LiDAR origin hits the cylindrical robot modelled as a circle at position (x_r, y_r) with radius r . The ray direction is:

$$(dx, dy) = (\cos \alpha_i, \sin \alpha_i)$$

The intersection is checked by substituting the parametric ray $P = \text{lidar_pos} + t \cdot (dx, dy)$ into the circle boundary — a point is on the circle when its distance to the center equals the radius, so we require $\|P - \text{robot_pos}\| = r$, or equivalently:

$$\|P - \text{robot_pos}\|^2 = r^2$$

This gives one equation in one unknown t — solve for t , then plug back in to get the hit point. Returns the closest hit (x_h, y_h) if $t > 0$ and within `lidar_range`, otherwise `None`.

Substituting the ray into the circle boundary and moving r^2 to the left:

$$\|P - \text{robot_pos}\|^2 - r^2 = 0$$

expands to a quadratic in t :

$$\underbrace{(dx^2 + dy^2)}_{a_c} t^2 + \underbrace{2(fx \cdot dx + fy \cdot dy)}_{b_c} t + \underbrace{(fx^2 + fy^2 - r^2)}_{c_c} = 0$$

where $(fx, fy) = \text{lidar_pos} - \text{robot_pos}$. Solving with the quadratic formula:

$$t = \frac{-b_c - \sqrt{b_c^2 - 4a_cc_c}}{2a_c}$$

The smallest positive t gives the closest hit point:

$$(x_h, y_h) = \text{lidar_pos} + t \cdot (dx, dy)$$

Noise model. Each hit point is converted to polar coordinates relative to the LiDAR origin — taking the difference in x and y between hit point and lidar gives the vector

between them, Pythagoras gives the true range and `atan2` gives the true bearing:

$$d = \sqrt{(x_r - x_l)^2 + (y_r - y_l)^2}, \quad \alpha = \text{atan2}(y_r - y_l, x_r - x_l)$$

where x_l, y_l is the lidar position and x_r, y_r the hit point on the robot.

Independent Gaussian noise is then added in polar domain:

$$\tilde{d} = d + \nu_d, \quad \nu_d \sim \mathcal{N}(0, \sigma_d^2) \quad \tilde{\alpha} = \alpha + \nu_\alpha, \quad \nu_\alpha \sim \mathcal{N}(0, \sigma_\alpha^2)$$

and converted back to Cartesian:

$$\tilde{x}_r = x_l + \tilde{d} \cos(\tilde{\alpha}), \quad \tilde{y}_r = y_l + \tilde{d} \sin(\tilde{\alpha})$$

Noise is added in polar domain because angle noise scales with distance — a 1° error at distance d causes a lateral shift of $d \cdot \sin(\nu_\alpha) \approx d \cdot \nu_\alpha$, while range noise σ_d is constant regardless of distance.

Why angle noise is more dangerous at long range. A 1° angle error causes a lateral shift of $\Delta = d \cdot \sin(\nu_\alpha)$:

$$d = 1 \text{ m} : \quad \Delta = 1 \cdot \sin(1^\circ) \approx 1 \cdot 0.017 = 1.7 \text{ cm}$$

$$d = 10 \text{ m} : \quad \Delta = 10 \cdot \sin(1^\circ) \approx 10 \cdot 0.017 = 17 \text{ cm}$$

Range noise σ_d is constant: 5 cm error at 1 m is the same 5 cm at 10 m.

Exercise 5: Robot in front of wall

We add a wall behind the robot. For each beam angle we check the robot first — if it hits, the wall behind is occluded:

$$\text{for each } \alpha_i : \quad \begin{cases} \text{check robot} \rightarrow \text{hit} & \rightarrow \text{record robot point, skip wall} \\ \text{check robot} \rightarrow \text{miss} & \rightarrow \text{check wall} \end{cases}$$

The wall intersection uses two parametric lines:

$$\text{ray: } P = \text{lidar} + t \cdot (dx, dy) \quad \text{wall: } W = \text{wall_start} + s \cdot (wx, wy)$$

Setting $P = W$ and solving gives t and s via the 2D cross product:

$$\text{denom} = dx \cdot wy - dy \cdot wx$$

If $\text{denom} = 0$ the ray is parallel to the wall — no intersection possible. Otherwise:

$$t = \frac{fx \cdot wy - fy \cdot wx}{\text{denom}}, \quad s = \frac{fx \cdot dy - fy \cdot dx}{\text{denom}}$$

where $(fx, fy) = \text{wall_start} - \text{lidar}$. Two conditions must both hold for a valid hit:

- $0 < t \leq \text{lidar_range}$ — in front of lidar and within range
- $s \in [0, 1]$ — within the actual wall segment, not the infinite line

Concrete example: lidar at $(0, 0)$, ray going right $(dx, dy) = (1, 0)$, wall from $(5, -3)$ to $(5, 3)$ so $(wx, wy) = (0, 6)$.

Finding t — how far along the ray:

$$\text{denom} = 1 \cdot 6 - 0 \cdot 0 = 6$$

$$fx = 5, \quad fy = -3$$

$$t = \frac{5 \cdot 6 - (-3) \cdot 0}{6} = 5 \quad \rightarrow \quad \text{hit at } (0, 0) + 5 \cdot (1, 0) = (5, 0) \checkmark$$

Finding s — where on the wall segment:

$$s = \frac{5 \cdot 0 - (-3) \cdot 1}{6} = 0.5 \quad \rightarrow \quad \text{halfway along wall } \checkmark$$

If hit at $(5, -4)$: $s < 0 \rightarrow \text{miss}$. If hit at $(5, 4)$: $s > 1 \rightarrow \text{miss}$.

Exercise 6: Wall cluster and line fitting

We fit a line $ax + by + c = 0$ to the wall point cloud by minimising the sum of squared perpendicular distances:

$$\min_{a,b,c} \sum_i \frac{(ax_i + by_i + c)^2}{a^2 + b^2}$$

where (a, b) is the normal vector defining line orientation and c the offset from the origin. The denominator $a^2 + b^2$ normalises to give true perpendicular distances. Solved numerically with `scipy.optimize.minimize`.

- $a = 1, b = 0 \rightarrow$ normal points in x -direction $\rightarrow$ line is **vertical**
- $a = 0, b = 1 \rightarrow$ normal points in y -direction $\rightarrow$ line is **horizontal**

Once a, b, c are found, solve for x to reconstruct the line for plotting:

$$ax + by + c = 0 \quad \rightarrow \quad x = \frac{-(by + c)}{a}$$

Describes how x changes as y moves along the line — for a perfect vertical wall $b \approx 0$ so $x \approx -c/a$ is constant, confirming the line is vertical. Evaluated at $y = [y_{\min}, y_{\max}]$ of the wall points — constraining the plotted line to the actual wall segment.

Example — vertical wall at $x = 5$: $a = 1, b = 0$ because the wall is perpendicular to the x -axis, $c = -5$ because the origin is 5 units from the wall:

$$\underbrace{a \cdot x}_{\text{how far in } x} + \underbrace{b \cdot y}_{\text{how far in } y} + \underbrace{c}_{\text{offset from origin}} = 0 \quad \rightarrow \quad 1 \cdot 5 + 0 \cdot y + (-5) = 0 \checkmark$$